Name of Student: VIKAS FOOLCHAND KUSHWAHA			
Roll Number: 29/B		**Lab Practical Number:** 03	
Title of LAB Practical: Create an application to demonstrate various Node.js Events in event emitter class.			
DOP: 01/10/2026		**DOS:** 05/10/2026	
CO Mapped: CO1	**PO Mapped:** PO3, PO5, PSO1, PSO2	**Faculty Signature:**	**Marks:**

Practical 1

Aim: Create an application to demonstrate various Node.js Events in event emitter class.

Description:

Node.js, being an asynchronous, event-driven runtime, is ideal for handling tasks that require non-blocking operations. The EventEmitter class, a core part of Node.js's events module, facilitates this programming paradigm by allowing the implementation of event-driven logic.

1. **Event-Driven Programming:** This paradigm involves the flow of program execution being determined by events such as user actions, sensor outputs, or messages from other programs. In Node.js, events are at the heart of non-blocking I/O, enabling efficient task handling.

2. **EventEmitter Class:** Provides an interface to create, listen to, and manage events. Includes methods such as on (for registering listeners) and emit (for triggering events).

3. **Asynchronous Operations:** Operations are performed in a non-blocking way, meaning other tasks can continue executing without waiting for the completion of previous tasks.

Implementation: Asynchronous Array Operations

Using EventEmitter, array operations such as sorting, reversing, and searching can be implemented as follows:

A. **Sorting:** Arranging elements in ascending or descending order.
B. **Reversing:** Changing the order of elements in the array.
C. **Searching:** Locating an element within the array and retrieving its index.

Advantages of this Approach

1. **Modularity:** Array operations are implemented as independent functions that are triggered by specific events.
2. **Reusability:** The same event listeners and functions can be reused for various operations.
3. **Non-blocking Execution:** Allows the main thread to remain free for other tasks while array operations are performed asynchronously.
4. **Scalability:** Additional operations can be added seamlessly by registering new events.

Applications: Real-time data processing, Chat applications (e.g., handling user events like "message sent" or "user joined"), Task schedulers or job queues, Event-driven frameworks like Express.js for building APIs.

Code:

QUESTION:
Create an application to demonstrate various Node.js Events in event emitter class. Create functions to sort, reverse and search for an element in an array. Register and trigger these functions using events.

CODE

```
const EventEmitter = require("events");

class ArrayEvents extends EventEmitter {
  sortArray(array) {
    const sortedArray = [...array].sort((a, b) => a - b);
    this.emit("sorted", sortedArray);
  }

  reverseArray(array) {
    const reversedArray = [...array].reverse();
    this.emit("reversed", reversedArray);
  }

  searchArray(array, element) {
    const index = array.indexOf(element);

    if (index !== -1) {
      this.emit("found", element, index);
    } else {
      this.emit("notFound", element);
    }
  }
}

const arrayEvents = new ArrayEvents();

// Register event listeners
arrayEvents.on("sorted", (result) => {
  console.log("Sorted array:", result);
});

arrayEvents.on("reversed", (result) => {
  console.log("Reversed array:", result);
});

arrayEvents.on("found", (element, index) => {
  console.log(`Element ${element} found at index ${index}.`);
});

arrayEvents.on("notFound", (element) => {
  console.log(`Element ${element} was not found in the array.`);
});

// Demonstrate a one-time event listener
arrayEvents.once("complete", () => {
```

CODE

```
  console.log("All array operations completed.");
});

// Demonstrate an event listener count
console.log(
  "Number of listeners for 'sorted':",
  arrayEvents.listenerCount("sorted")
);

const numbers = [8, 3, 10, 1, 6, 4];

// Trigger events by calling the methods
arrayEvents.sortArray(numbers);
arrayEvents.reverseArray(numbers);
arrayEvents.searchArray(numbers, 6);
arrayEvents.searchArray(numbers, 20);

// Trigger the completion event
arrayEvents.emit("complete");

// This will not execute because "complete" was registered with
once()
arrayEvents.emit("complete");
```

Output:

```
> node events.js
Number of listeners for 'sorted': 1
Sorted array: [ 1, 3, 4, 6, 8, 10 ]
Reversed array: [ 4, 6, 1, 10, 3, 8 ]
Element 6 found at index 4.
Element 20 was not found in the array.
All array operations completed.
```

Conclusion:

This practical demonstrates the flexibility and power of Node.js's `EventEmitter` class to handle asynchronous operations and modularize functionality. By registering and triggering events, we decouple logic into reusable functions like sorting, reversing, and searching an array. This approach enhances code readability, scalability, and maintainability, making it ideal for real-world applications involving event-driven programming.